comprehensions work.[2] In the snippet that follows, what will be the output of the last line? You might be surprised at the answer; try it on your REPL:

[2] I made one of these mistakes while preparing the listings for this very chapter.

```
people = [ 'bill', 'ted' ]
greetings = {}

for person in people
  greetings[person] = ->
    "My name is #{person}"

greetings.bill()
```

Why does the final expression here evaluate to `'My name is ted'`? You see, functions have access to variables via lexical scope regardless of when they're invoked. Here, there's only one `person` variable in scope. Once the comprehension has run, it will contain the last value assigned to it and not the value it had when the function was declared.

If you *really* need to define a function inside a comprehension and have it access some value from inside the comprehension, then you'll need to create a new lexical scope with another function:

```
people = [ 'bill', 'ted' ]
greetings = {}

for person in people
  do ->
    name = person
    greetings[name] = ->
      "My name is #{name}"

greetings.bill()
# My name is bill
```

Now the function(s) assigned to greetings[name] will close over the name variables created for each one. Revisit chapter 3 later if you're still not entirely comfortable with closures and how they close over variables.

In contrast, if you use a `forEach` for this example, then you avoid the problem because you're creating a function scope by default:

```
people.forEach (name) -> greetings[name] = "My name is #{name}"
```

A comprehension that needs a scope probably shouldn't be a comprehension.